

- 01 — Cloud Run: Deploying Containerized Applications
 - Exam Relevance
 - 1. What Is Cloud Run?
 - Two flavors
 - 2. The Container Contract
 - 3. Deploying a Service
 - From a container image (Artifact Registry)
 - From source (Cloud Build builds the image)
 - Console path
 - 4. Key Deployment Parameters
 - 5. Revisions & Traffic Management
 - 6. Authentication & Access Control
 - 7. Scaling Behavior
 - 8. Networking
 - VPC egress (calling private resources)
 - Ingress
 - 9. Secrets & Config
 - 10. Triggering Cloud Run from Events
 - a) Pub/Sub push subscription
 - b) Eventarc trigger (recommended)
 - c) Cloud Scheduler (cron)
 - 11. Cloud Run Jobs (vs. Services)
 - 12. Common gcloud Operations
 - 13. Exam Traps & Keywords
 - 14. Sources

01 — Cloud Run: Deploying Containerized Applications

Exam Relevance

You must be able to deploy a container to Cloud Run, configure traffic, manage revisions, control access, and connect to other GCP services. **Cloud Run questions appear repeatedly on the ACE exam.**

1. What Is Cloud Run?

- **Fully managed serverless platform** for running stateless **containers**
- Scales from **0** to thousands of instances automatically
- Pay per **request + CPU/memory used** (rounded to nearest 100 ms)
- HTTPS endpoint provisioned automatically (***.run.app** domain)
- Supports **HTTP/1, HTTP/2, gRPC, WebSockets**

Two flavors

	Cloud Run (managed)	Cloud Run on GKE (Anthos)
Infrastructure	Google-managed	Your own GKE cluster
Use case	Default — fully serverless	Hybrid / on-prem requirements

Exam default: unless told otherwise, "Cloud Run" = fully managed.

2. The Container Contract

Your container must:

1. Listen on the **port** defined by the **PORT** environment variable (default: **8080**)
2. Listen on **0.0.0.0** (not **127.0.0.1**)
3. Start within **4 minutes** (otherwise the deployment fails)
4. Be **stateless** — local disk is ephemeral; no file persists between requests

3. Deploying a Service

From a container image (Artifact Registry)

```
gcloud run deploy my-service \  
  --image=us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1 \  
  --region=us-central1 \  
  --platform=managed
```

```
--platform=managed \
--allow-unauthenticated \
--port=8080 \
--memory=512Mi \
--cpu=1 \
--min-instances=0 \
--max-instances=100 \
--timeout=300 \
--concurrency=80 \
--set-env-vars="ENV=prod,DB_HOST=10.0.0.5" \
--service-account=my-sa@PROJECT_ID.iam.gserviceaccount.com
```

From source (Cloud Build builds the image)

```
gcloud run deploy my-service \
  --source=. \
  --region=us-central1 \
  --allow-unauthenticated
```

Cloud Build automatically uses **Buildpacks** if there is no **Dockerfile**.

Console path

Cloud Run → Create Service → Deploy one revision from existing container image / source repo

4. Key Deployment Parameters

Parameter	Description	Default
<code>--image</code>	Container image URL	required
<code>--region</code>	Region	required
<code>--port</code>	Port your container listens on	8080
<code>--memory</code>	Memory per instance	512Mi
<code>--cpu</code>	vCPU count (1, 2, 4, 6, 8)	1

Parameter	Description	Default
<code>--min-instances</code>	Always-warm instances (avoids cold start)	0
<code>--max-instances</code>	Cap on concurrent instances	100
<code>--concurrency</code>	Max simultaneous requests per instance	80
<code>--timeout</code>	Max request duration in seconds	300 (max 3600)
<code>--allow-unauthenticated</code>	Public access	requires auth by default
<code>--ingress</code>	<code>all</code> / <code>internal</code> / <code>internal-and-cloud-load-balancing</code>	<code>all</code>
<code>--service-account</code>	Identity used by the service	Default Compute SA
<code>--set-env-vars</code>	Plain env vars	none
<code>--set-secrets</code>	Mount secrets from Secret Manager	none
<code>--vpc-connector</code>	Serverless VPC Access connector	none
<code>--vpc-egress</code>	<code>all-traffic</code> / <code>private-ranges-only</code>	private-ranges-only
<code>--cpu-boost</code>	Boost CPU during cold start	off

5. Revisions & Traffic Management

Every deploy creates a new **revision** (immutable). Revisions stay around so you can roll back or split traffic.

```
# List revisions
gcloud run revisions list --service=my-service --region=us-central1

# Send 100% to a specific revision
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00005-abc=100

# Canary: 90/10 split between revisions
```

```
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00004-xyz=90,my-service-00005-abc=10

# Roll back
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00004-xyz=100
```

You can also tag revisions and route to them by URL:

```
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --set-tags=preview=my-service-00005-abc
# Now reachable at: https://preview---my-service-xxxxx-uc.a.run.app
```

6. Authentication & Access Control

Mode	How
Public	<code>--allow-unauthenticated</code> (grants <code>allUsers</code> the <code>run.invoker</code> role)
Authenticated callers only	<code>--no-allow-unauthenticated</code> (default)
Specific user / SA	<code>gcloud run services add-iam-policy-binding ...</code> <code>--role=roles/run.invoker</code>

To call an authenticated service:

```
curl -H "Authorization: Bearer $(gcloud auth print-identity-token)" \
  https://my-service-xxxxx-uc.a.run.app
```

Key IAM roles:

- `roles/run.invoker` — call the service
 - `roles/run.developer` — deploy & manage
 - `roles/run.admin` — full control
-

7. Scaling Behavior

- **min-instances=0** → scales to zero, cheapest, but **cold starts** on first request
- **min-instances=1+** → no cold start, you pay even when idle
- **max-instances** → protects downstream systems from being overwhelmed
- **concurrency** → high concurrency (e.g., 80) keeps fewer instances; concurrency=1 mimics traditional one-request-per-VM model

Exam tip: When asked how to **eliminate cold starts**, the answer is `--min-instances >= 1`.

8. Networking

VPC egress (calling private resources)

- **Direct VPC egress** (newer, recommended): `--network` + `--subnet`
- **Serverless VPC Access connector:** `--vpc-connector`

```
gcloud run deploy my-service \  
  --image=... \  
  --vpc-connector=my-connector \  
  --vpc-egress=all-traffic
```

Ingress

- **all** — open internet
 - **internal** — only VPC + projects in same VPC SC perimeter
 - **internal-and-cloud-load-balancing** — only via External HTTPS LB (used to attach Cloud Armor / IAP)
-

9. Secrets & Config

```
# Mount Secret Manager secret as env var
gcloud run deploy my-service \
  --image=... \
  --set-secrets=DB_PASSWORD=db-password:latest

# Or as a file
gcloud run deploy my-service \
  --image=... \
  --set-secrets=/etc/secrets/db=db-password:latest
```

The service's runtime SA must have `roles/secretmanager.secretAccessor`.

10. Triggering Cloud Run from Events

Cloud Run is the **target** for many event sources. Three common patterns:

a) Pub/Sub push subscription

```
gcloud pubsub subscriptions create my-sub \
  --topic=my-topic \
  --push-endpoint=https://my-service-xxxxx-uc.a.run.app/ \
  --push-auth-service-account=invoker-sa@PROJECT_ID.iam.gserviceaccount.com
```

The push endpoint receives an HTTP POST with the message in JSON.

b) Eventarc trigger (recommended)

See `06-eventarc.md`. Lets you wire any GCP event source.

c) Cloud Scheduler (cron)

```
gcloud scheduler jobs create http nightly-job \
  --schedule="0 2 * * *" \
  --uri=https://my-service-xxxxx-uc.a.run.app/run \
  --oidc-service-account-email=scheduler-
sa@PROJECT_ID.iam.gserviceaccount.com
```

11. Cloud Run Jobs (vs. Services)

- **Service** — long-running, request-driven (HTTP)
- **Job** — runs to completion (batch). No HTTP endpoint. Up to **24-hour** runtime per task, parallel tasks supported.

```
gcloud run jobs create my-job \  
  --image=us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-batch:v1 \  
  --region=us-central1 \  
  --tasks=10 \  
  --parallelism=5 \  
  --task-timeout=3600
```

```
gcloud run jobs execute my-job --region=us-central1
```

Exam tip: "Run a batch script that finishes in N minutes/hours" → **Cloud Run Job**, not Service.

12. Common gcloud Operations

```
# Update env vars without redeploy  
gcloud run services update my-service \  
  --region=us-central1 \  
  --update-env-vars=NEW_FLAG=true  
  
# Describe service  
gcloud run services describe my-service --region=us-central1  
  
# Get URL  
gcloud run services describe my-service --region=us-central1 \  
  --format="value(status.url)"  
  
# Delete  
gcloud run services delete my-service --region=us-central1
```

13. Exam Traps & Keywords

If you see...	Answer
"fully managed, container, scale to zero"	Cloud Run
"eliminate cold starts"	<code>--min-instances >= 1</code>
"stop public access"	<code>--no-allow-unauthenticated</code> + IAM <code>run.invoker</code>
"canary / split traffic"	revisions + <code>update-traffic</code>
"private ingress only via internal LB"	<code>--ingress=internal-and-cloud-load-balancing</code>
"long-running batch task"	Cloud Run Job , not Service
"Cloud Run can talk to Cloud SQL on private IP"	Use VPC connector / Direct VPC + Cloud SQL Proxy

14. Sources

- [Cloud Run overview](#)
- [Deploying services](#)
- [Managing traffic](#)
- [Cloud Run Jobs](#)
- [Connecting to a VPC](#)